

SMARTFOLIO
PERSONAL PORTFOLIO & TASK MANAGEMENT
WEB APPLICATION

A Project Report
On Full Stack Development Fundamentals

TECHNOLOGIES USED

React.js | Node.js | Express.js | MySQL Database
HTML | CSS | JavaScript | Postman

Submitted by
YARASURI V N V SAI SRI TEJASWI
Master of Computer Applications
(MCA)

Submitted to
NeoSkillz



Academic Year
2025 – 2026

Contents

1. INTRODUCTION.....	4
2. PROJECT OBJECTIVE.....	5
3. PROJECT OVERVIEW	6
4. TECHNOLOGIES USED	7
4.1 Frontend Technologies	7
4.2 Backend Technologies	7
4.3 Database	7
5. SYSTEM REQUIREMENTS	9
5.1 HARDWARE REQUIREMENTS	9
5.2 SOFTWARE REQUIREMENTS	9
6. PROJECT ARCHITECTURE	10
7. FRONTEND DEVELOPMENT	12
8. BACKEND DEVELOPMENT	15
9. DATABASE CONNECTIVITY	18
10. API TESTING USING POSTMAN	20
11. GITHUB REPOSITORY LINK.....	26
12. DEPLOYMENT LINK.....	27
13. TEST CASES	28
14. CHALLENGES FACED	29
15. CONCLUSION	30

LIST OF FIGURES

Figure 1: Components of Full Stack Development.....	4
Figure 2: Overview of Personal Portfolio & Task Management Web Application	6
Figure 3:Tools and Technologies Used	8
Figure 4: Application Flow Diagram of Personal Portfolio & Task Management Web Application...	11
Figure 5: Home Page	12
Figure 6: Portfolio-Home Page	13
Figure 7: Portfolio- About Page.....	13
Figure 8: Login Page.....	14
Figure 9: Register Page.....	14
Figure 10: server.js file	15
Figure 11: db.js file	16
Figure 12: API Routes	16
Figure 13: Tasks Dashboard	17
Figure 14: Student Tasks Dashboard	17
Figure 15: MySQL Database connection.....	18
Figure 16: Users Table.....	19
Figure 17: Tasks Table.....	19
Figure 18: Testing User Registration API using POST Method.....	21
Figure 19: Testing User Login API using POST Method.....	21
Figure 20: Retrieve Users Using GET Method.....	22
Figure 21: Testing Forgot Password using PUT Method.....	22
Figure 22: Retrieve Tasks	23
Figure 23: Add Task to the user.....	23
Figure 24: Update Task.....	24
Figure 25: Delete Task.....	24
Figure 26: Task Management Dashboard Interface	25
Figure 27: Updated Users Table	25
Figure 28: Updated Tasks Table	25
Figure 29: Test Cases and Results	28
Figure 30: Postman API Testing Summary	28

1. INTRODUCTION

The Personal Portfolio & Task Management Web Application is a full stack web development project undertaken as part of the NeoSkillz Full Stack Development Internship Program. The purpose of building this project was to practically implement the ideas and theories that were learned during the training phase like front-end development, back-end integration, Database connectivity, REST API handling, deployment etc. This application comprises a professional personal portfolio page and a task management system that helps in adding, updating, deleting and managing the tasks. React.js, HTML, CSS and JavaScript were used for front-end to build a responsive and interactive interface whereas, Node.js and Express.js was used for back-end development. MySQL database was used to save the data regarding the tasks. RESTful APIs were developed for CRUD operations on the database and to test the same postman was used. The full stack web application was successfully developed and knowledge about integrating front-end, back-end, Database and APIs was gained.

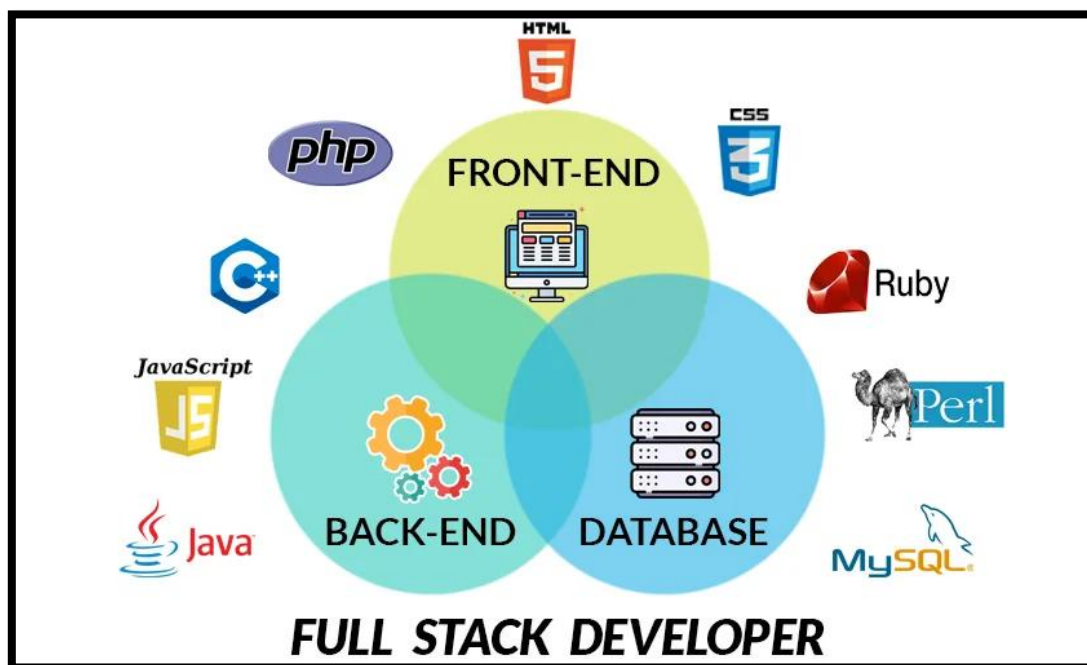


Figure 1: Components of Full Stack Development

2. PROJECT OBJECTIVE

The primary aim of the project is to create a Personal Portfolio & Task Management Web Application with the help of full stack development technologies. This project helps to get practical knowledge about frontend development, backend integration, connecting database with applications and how to create a REST API and utilize it to build a Personal Portfolio & Task Management Web Application which displays details about portfolio such as Skills, projects, certifications, contact information, etc., and task management system, so that users can perform the following operations- add tasks, update, delete tasks efficiently. This project helps to develop hands-on knowledge about creating a responsive User Interface using the React.js framework and developing server-side logic with the Node.js and Express.js framework. This project further allows one to learn about CRUD operation, Postman to test API, project structure and the readiness of the project for deployment.

Key Objectives:

- Create a responsive and user-friendly personal portfolio website.
- Display skills, projects, credentials, and contacts effectively.
- Integrate task management functionality to manage tasks efficiently.
- Enable users to add, modify, delete, and manage tasks.
- Use RESTful APIs to conduct CRUD operations.
- Integrate the front end, back end, and database smoothly.
- Design an interactive user interface by using React.js.
- Implement the server functionality using Node.js and Express.js.
- Test the APIs using Postman.
- Make the application ready for deployment and actual usage.

3. PROJECT OVERVIEW

The Personal Portfolio & Task Management Web Application is meant to offer a way of showcasing the personal portfolio in a neat and systematic fashion as well as manage tasks for the day more efficiently. Through the portfolio part, a user can showcase the details like technical skills, projects they've done, certificates and contact details to help show a professional display on the web. With the module for the portfolio comes the task managing part of the application that enables us to create, edit and delete a task. The system is built with an attractive, clean and a responsive interface for all kind of usage. It can be used for communication with backend by using the REST APIs, with database for storing all the task related details. The project makes sense by implementing the core of the full stack web development.

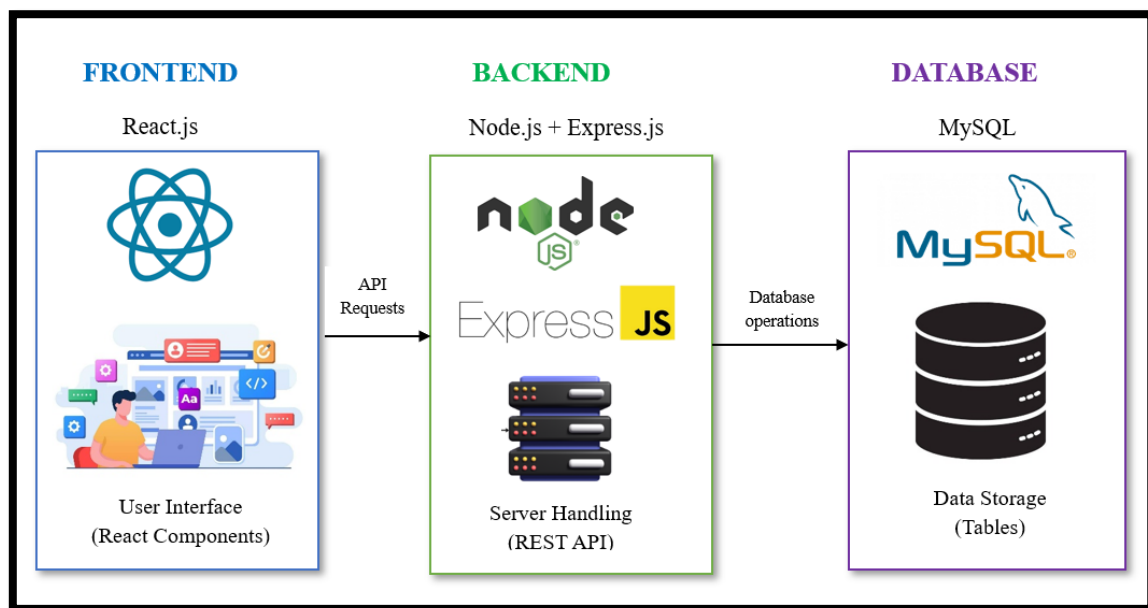


Figure 2: Overview of Personal Portfolio & Task Management Web Application

4. TECHNOLOGIES USED

The Personal Portfolio & Task Management Web Application was built using several modern full-stack technologies.

These technologies include:

4.1 Frontend Technologies

The frontend of the application was built with React.js combined with HTML, CSS, and JavaScript.

- React.js was utilized for reusable components and for building the responsive interface,
- HTML was responsible for the structure of the page,
- CSS was for styling and positioning of elements, and
- JavaScript was for the client-side functionalities and interactivity.

4.2 Backend Technologies

For backend of the application, Node.js and Express.js was utilized.

- Node.js was to execute the server-side actions,
- Express.js was to be used for routing, middleware and building RESTful API.
- The backend of the application would handle communication between frontend and database, handle tasks creating, updating, deleting and getting through APIs.

4.3 Database

MySQL was chosen as a Database Management System (DBMS) used to store and maintain data related to the tasks. MySQL aids in structuring data in table format, and efficiently performing operations like insert, update, delete and retrieve data. Database was interfaced with backend server for seamless communication with application and to retrieve/modify stored data. Reliable and rapid execution makes MySQL a suitable database for Node.js and Express.js application.

The application was tested using CRUD (Create, Read, Update and Delete) operations to verify its correct data processing and overall working. The REST API were tested using the postman to test the server and its responses as well as for checking whether it is functioning and communicating between front-end and back-end properly or not, including all API methods like get, post, put, delete to verify their function in the task managing application. For coding and managing the project, Visual Studio Code was the mainly used IDE, and for managing the codes and maintaining the project along with tracking updates, GitHub was used.

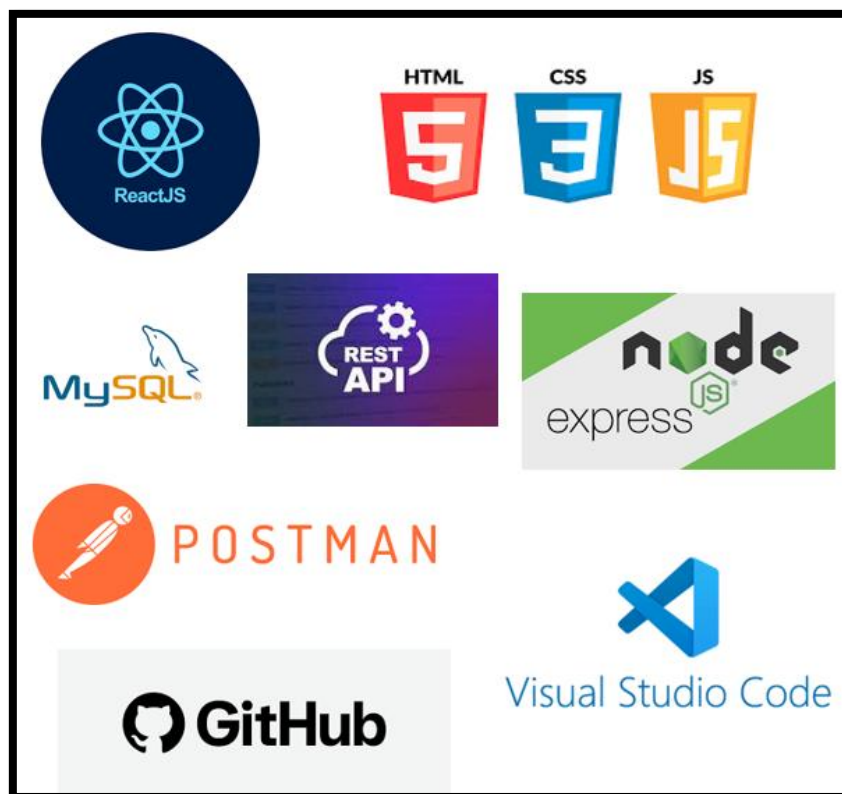


Figure 3:Tools and Technologies Used

5. SYSTEM REQUIREMENTS

Following hardware and software requirements are needed for the effective development and execution of Personal Portfolio & Task Management Web Application, so that the application runs efficiently without any problem during development, can test APIs and work without data connection.

5.1 HARDWARE REQUIREMENTS

Processor: Intel i3 or more

RAM: Minimum 4 GB

Storage: Minimum 20GB Free Space

System Type: 64-bit Computer

Internet: Needed during the deployment and also if need to get access of Git-Hub.

5.2 SOFTWARE REQUIREMENTS

OS: Windows 10/11

Frontend: React.js, HTML, CSS, JavaScript

Backend: Node.js, Express.js

Database: MySQL

IDE: Visual Studio Code

API testing Tool: Postman

Version Control: Git-Hub

Browser: Google Chrome/Microsoft Edge

6. PROJECT ARCHITECTURE

The architecture of the Personal Portfolio & Task Management Web Application uses full-stack three tier architecture, i.e. A three-layered structure comprises of Frontend, Backend and Database as illustrated above. The frontend layer uses React.js with HTML, CSS and JavaScript for making the application interactive and responsive to show portfolio details and managing tasks. The backend layer uses Node.js with Express.js for handling server-side, routing requests and all the REST API calls and the backend acts as a middle layer between the frontend and the database. The data layer, MySQL database, is used to store the data of tasks in secure manner. All the communications between the frontend and the backend are done through REST API and between backend and the database, through CRUD operations.

APPLICATION FLOW

1. The user logs in to Personal Portfolio & Task Management Web Application through the web browser.
2. The portfolio and task management interface is presented to the user through the frontend developed by React.js.
3. On interaction with the interface, the frontend sends an HTTP request to the backend through REST API for different actions like Add Task, Update Task, view tasks and delete tasks.
4. The business logic, processing and data validation of the request is done by the backend developed using Node.js, Express.js framework.
5. The backend communicates with the MySQL database to add/delete/update/fetch task information.
6. The MySQL database receives the operations from the backend, perform it accordingly and send the relevant data back to the backend.
7. A relevant response (JSON format) is sent back from the backend to the frontend.
8. The frontend update its interface dynamically and the latest task information as well as portfolio information is presented to the user.

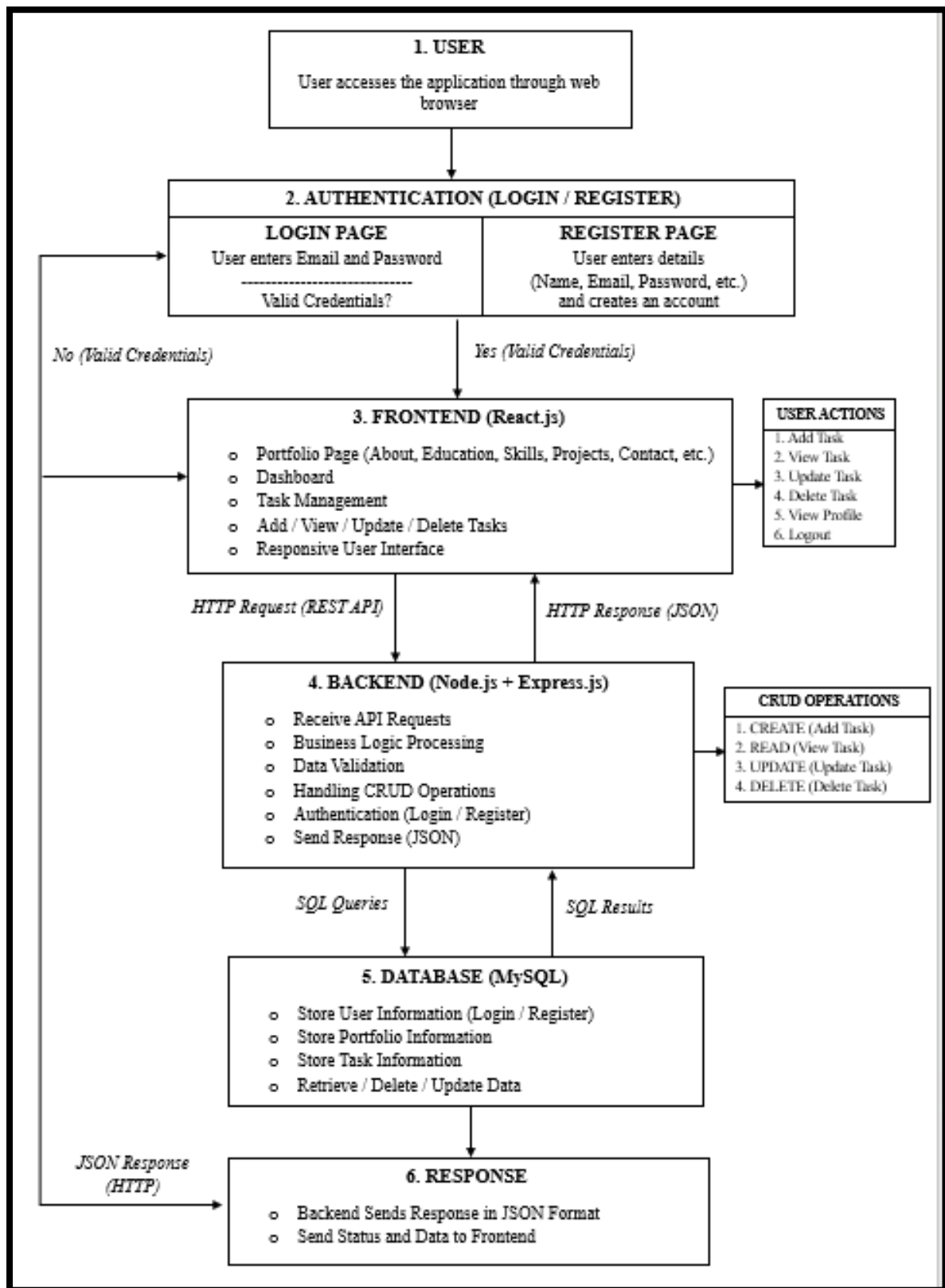


Figure 4: Application Flow Diagram of Personal Portfolio & Task Management Web Application

7. FRONTEND DEVELOPMENT

The frontend of the Personal Portfolio & Task Management Web Application has been designed using the HTML, CSS, JavaScript and the React.js library to build up responsive & interactive User interface. The components were developed and assembled to form a user interface using React.js. A neat and user-friendly layout was maintained to navigate easily across different devices and make the interface more appealing and easier to use. The whole interface was broken down into multiple sections:

Home, About, Education, Skills, Internships, Projects, Certifications, Contact, Login, Register, and Task Management.

The interface for task management helps users add, update, view and delete task details using the concept of REST APIs. The CSS has been implemented to design and style the application and the JavaScript has been used to develop the functionalities. The HTTP requests are sent by the frontend from where the information received from the backend is displayed to the user without reloading the entire webpage.

Home.js:

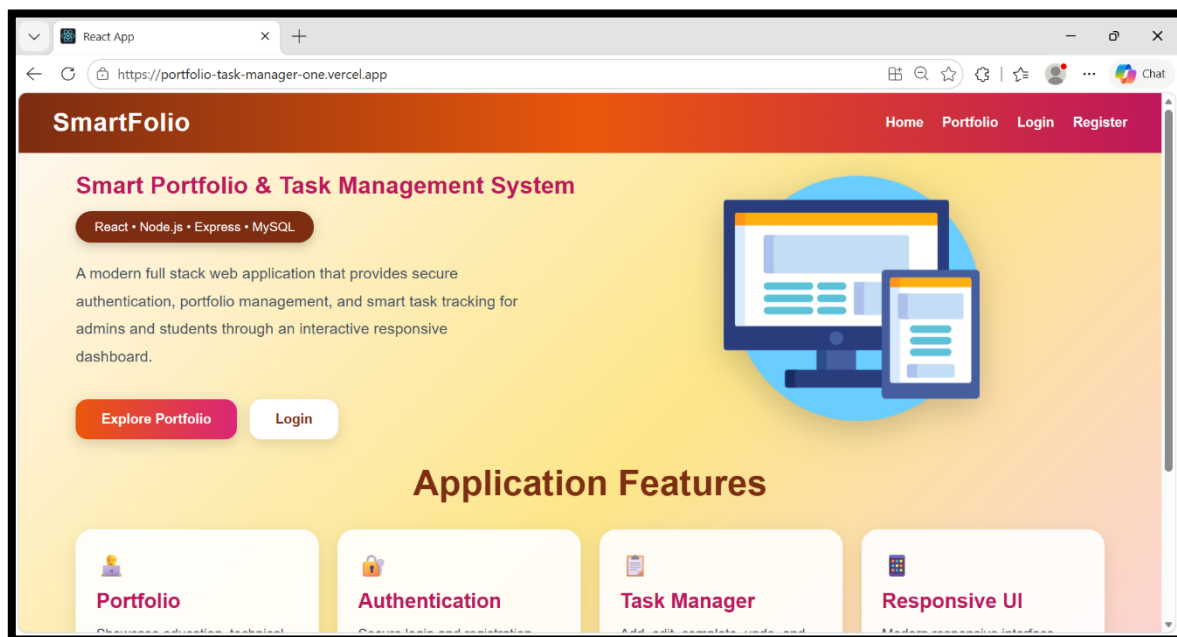


Figure 5: Home Page

Portfolio.js:

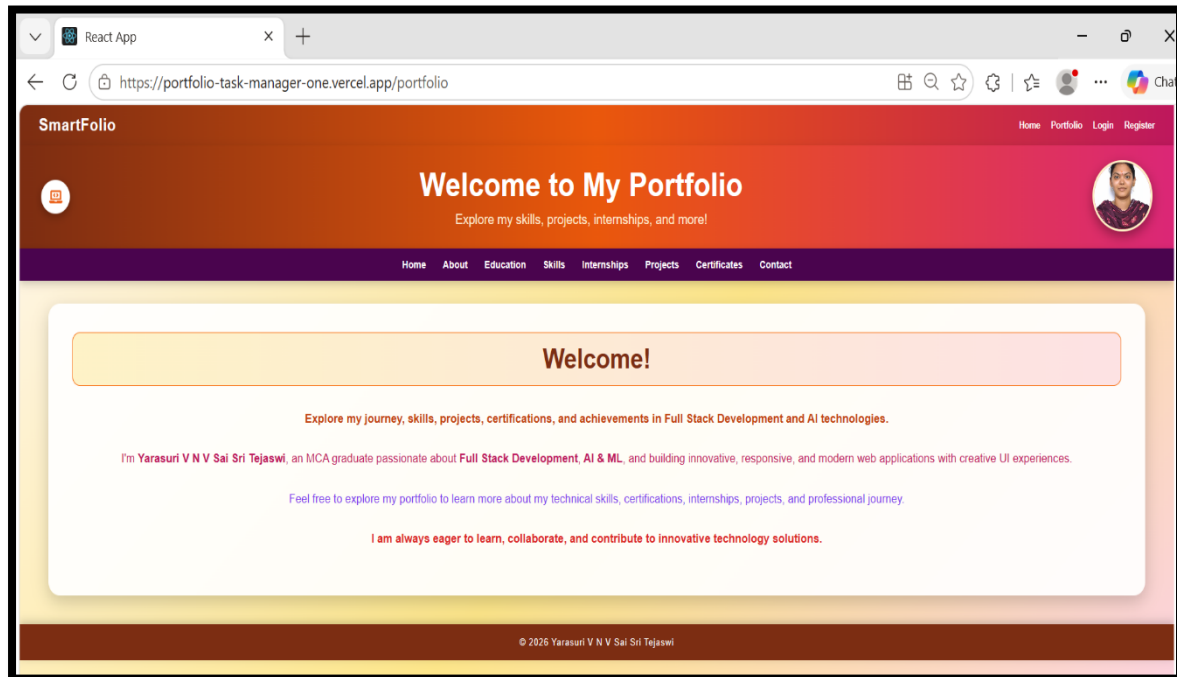


Figure 6: Portfolio-Home Page

About.js:

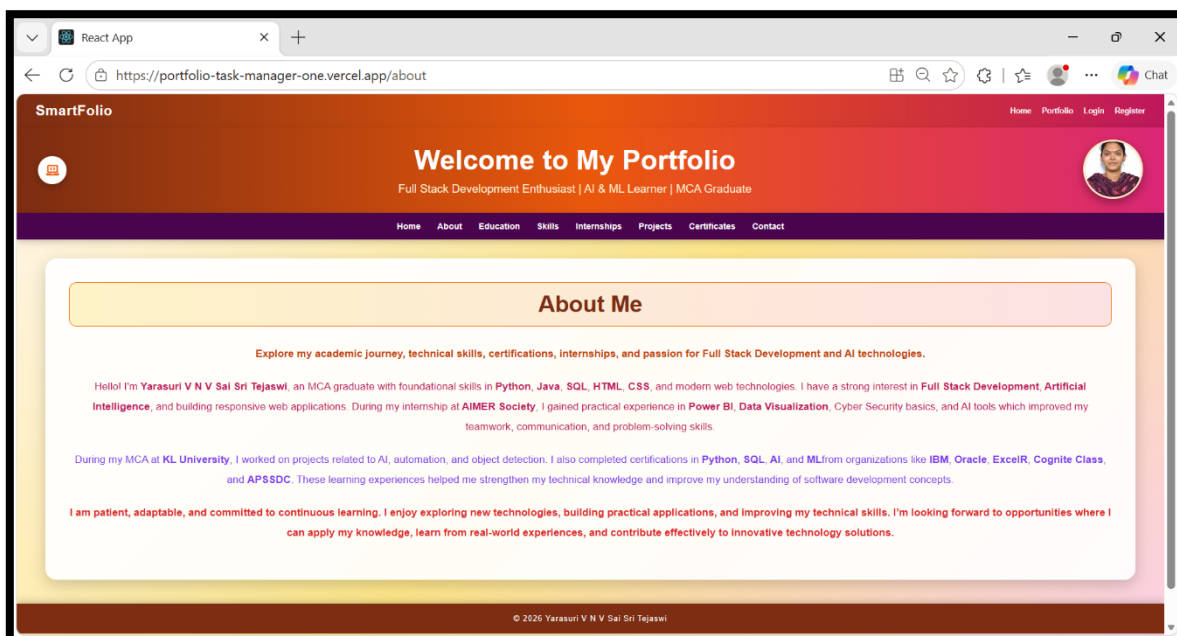
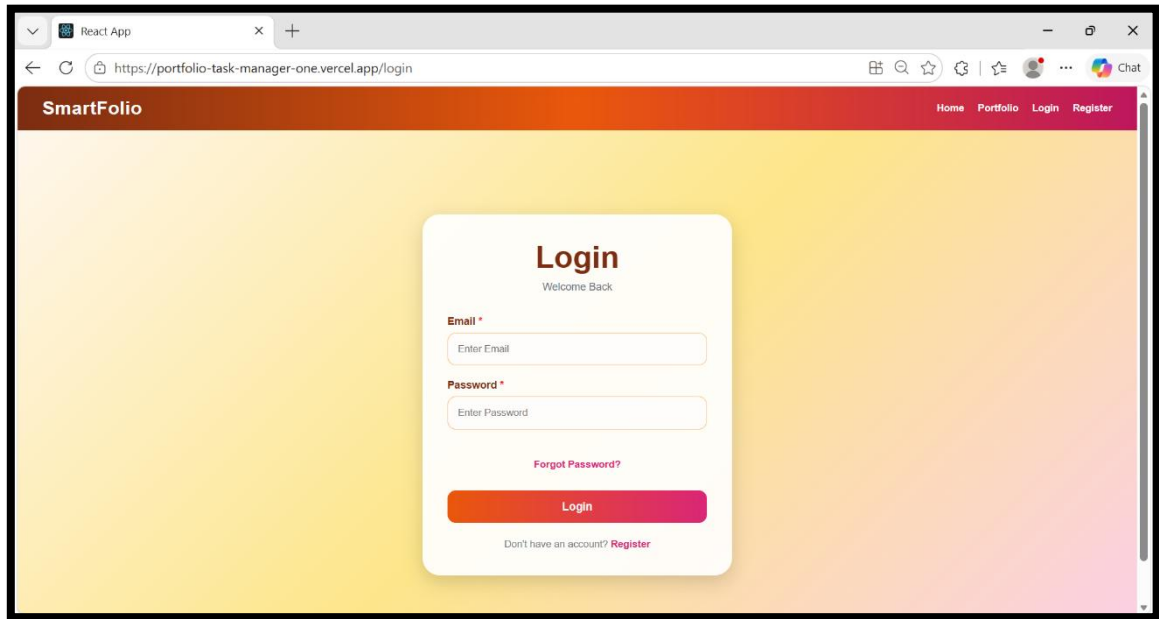


Figure 7: Portfolio- About Page

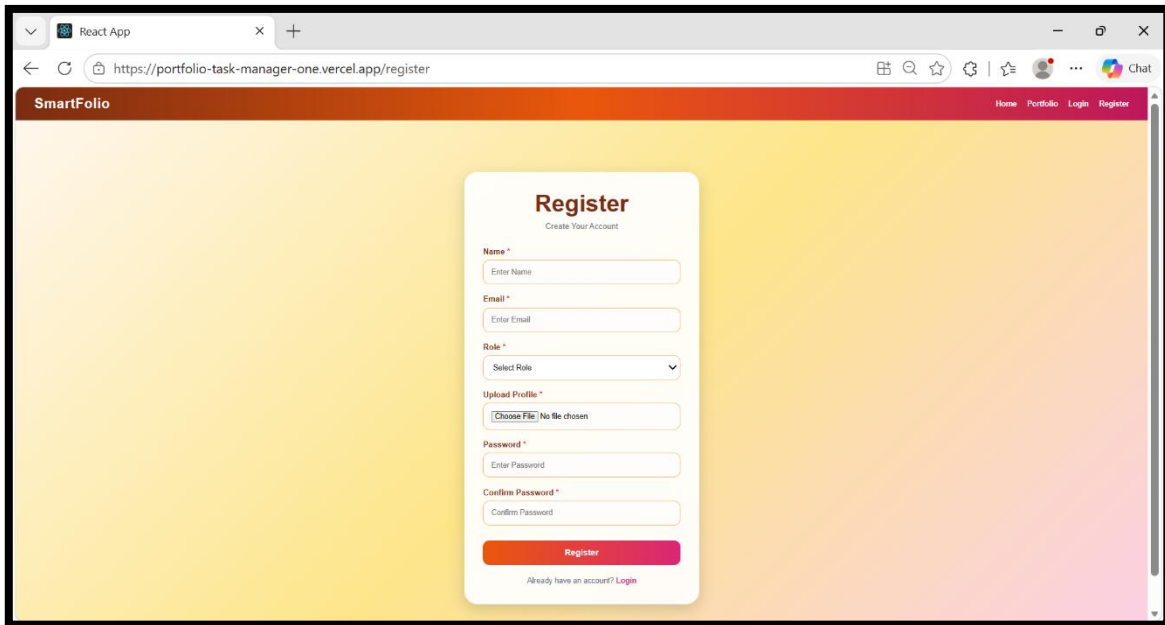
Login.js:



The screenshot shows a web browser window with the URL `https://portfolio-task-manager-one.vercel.app/login`. The page features a dark red header with the "SmartFolio" logo and navigation links for "Home", "Portfolio", "Login", and "Register". The main content area has a light yellow-to-pink gradient background. In the center is a white login card with the title "Login" and the subtitle "Welcome Back". The card contains two input fields labeled "Email *" and "Password *", each with placeholder text "Enter Email" and "Enter Password" respectively. Below these fields is a link for "Forgot Password?". At the bottom of the card is a red "Login" button and a link for "Don't have an account? Register".

Figure 8: Login Page

Register.js:



The screenshot shows a web browser window with the URL `https://portfolio-task-manager-one.vercel.app/register`. The page features a dark red header with the "SmartFolio" logo and navigation links for "Home", "Portfolio", "Login", and "Register". The main content area has a light yellow-to-pink gradient background. In the center is a white register card with the title "Register" and the subtitle "Create Your Account". The card contains several input fields: "Name *" (placeholder "Enter Name"), "Email *" (placeholder "Enter Email"), "Role *" (a dropdown menu with "Select Role" and a downward arrow), "Upload Profile *" (a file upload button with "Choose File" and "No file chosen"), "Password *" (placeholder "Enter Password"), and "Confirm Password *" (placeholder "Confirm Password"). At the bottom of the card is a red "Register" button and a link for "Already have an account? Login".

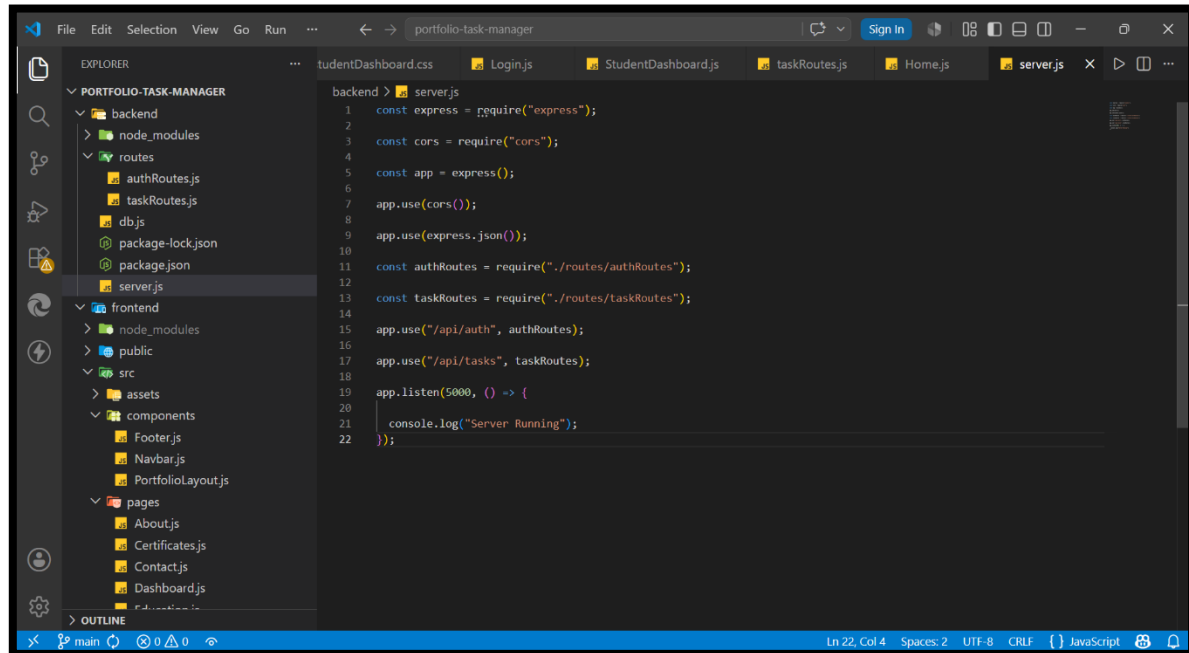
Figure 9: Register Page

8. BACKEND DEVELOPMENT

The backend of the Personal Portfolio & Task Management Web Application was built using Node.js & Express.js which were well suited for the task of server management & API creation. Node.js served as the environment to run the JavaScript on the server side and also acted as a tool to design rest-full APIs and manage the various requests and responses and route traffic effectively between the frontend and the MySQL database.

The backend was used to develop, implement & design the various CRUD operation used to add, get, delete, update the data associated with tasks and also handle the authentication mechanisms for registering and login of users. The backend had to design and expose rest-full APIs that accept the request from the front-end and return appropriate responses in JSON. It had to handle and check data validity and error reporting for effective & secure data management. Postman was used to test the API functions.

server.js:

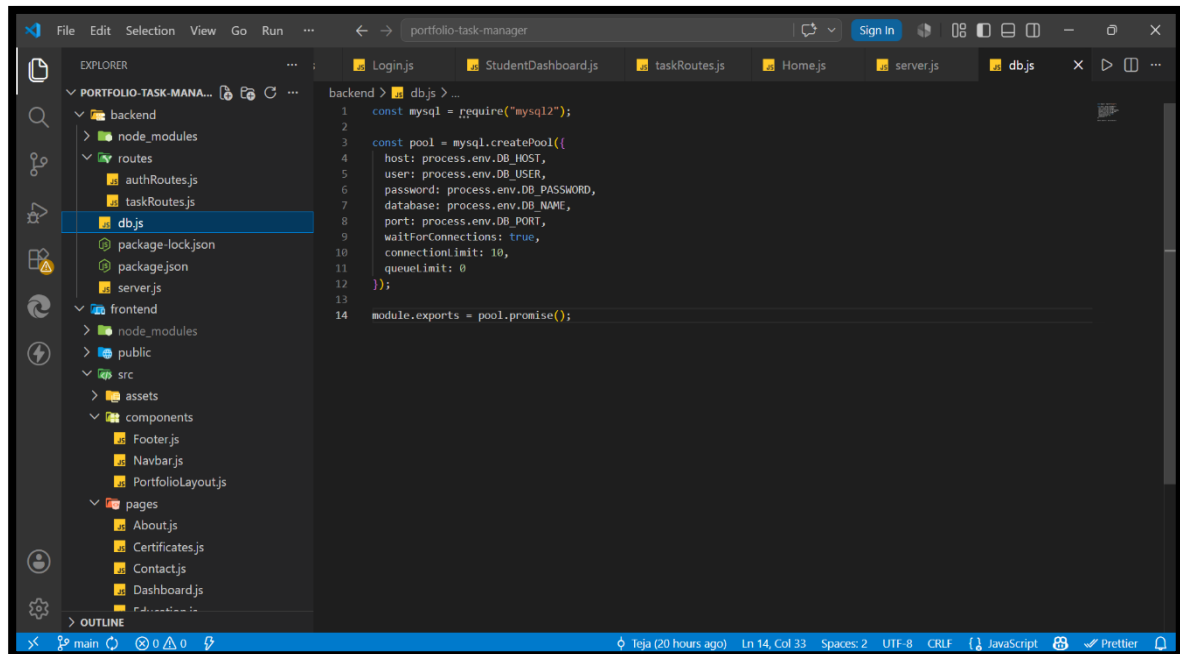
The image shows a screenshot of a code editor interface. The top bar indicates the file path 'portfolio-task-manager'. The left sidebar shows a file explorer with a tree structure: 'PORTFOLIO-TASK-MANAGER' (expanded) contains 'backend' (expanded) with 'node_modules', 'routes' (expanded) with 'authRoutes.js', 'taskRoutes.js', 'db.js', 'package-lock.json', 'package.json', and 'server.js' (selected), 'frontend' (expanded) with 'node_modules', 'public', 'src', 'assets', 'components' (expanded) with 'Footer.js', 'Navbar.js', 'PortfolioLayout.js', and 'pages' (expanded) with 'About.js', 'Certificates.js', 'Contact.js', and 'Dashboard.js'. The main editor area shows the content of 'server.js' with the following code:

```
1 const express = require("express");
2 const cors = require("cors");
3
4 const app = express();
5
6 app.use(cors());
7
8 app.use(express.json());
9
10 const authRoutes = require("../routes/authRoutes");
11 const taskRoutes = require("../routes/taskRoutes");
12
13 app.use("/api/auth", authRoutes);
14 app.use("/api/tasks", taskRoutes);
15
16 app.listen(5000, () => {
17   console.log("Server Running");
18 });
```

The bottom status bar shows 'Ln 22, Col 4', 'Spaces: 2', 'UTF-8', 'CRLF', and 'JavaScript'.

Figure 10: server.js file

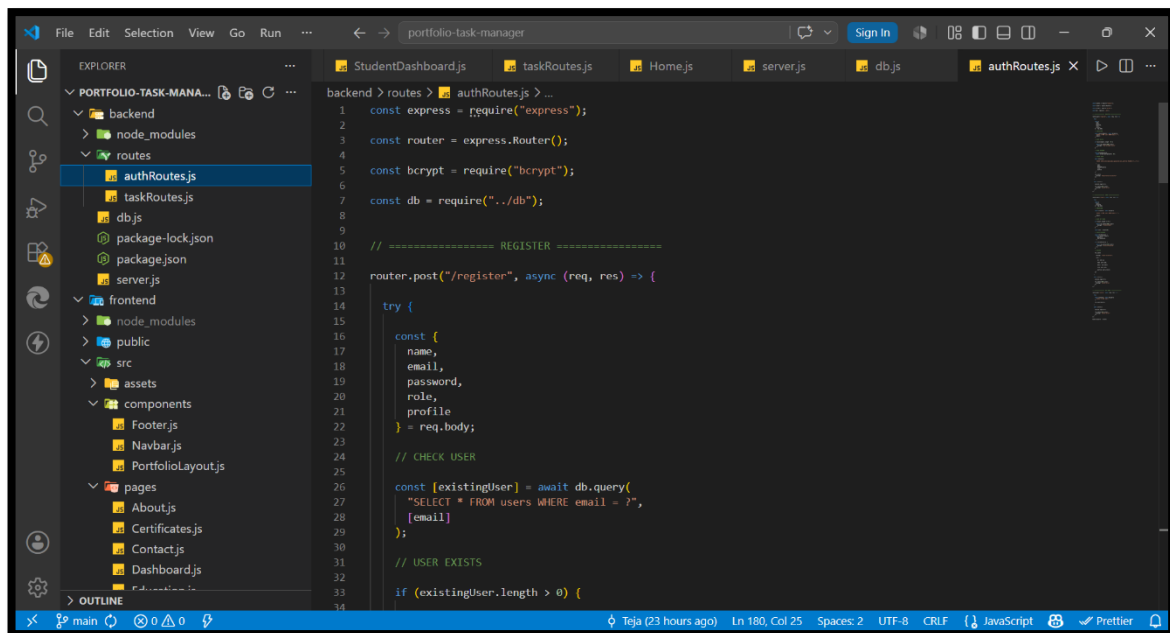
db.js:



```
1 const mysql = require("mysql2");
2
3 const pool = mysql.createPool({
4   host: process.env.DB_HOST,
5   user: process.env.DB_USER,
6   password: process.env.DB_PASSWORD,
7   database: process.env.DB_NAME,
8   port: process.env.DB_PORT,
9   waitForConnections: true,
10  connectionLimit: 10,
11  queueLimit: 0
12 });
13
14 module.exports = pool.promise();
```

Figure 11: db.js file

authRoutes.js:



```
1 const express = require("express");
2
3 const router = express.Router();
4
5 const bcrypt = require("bcrypt");
6
7 const db = require("../db");
8
9
10 // ===== REGISTER =====
11
12 router.post("/register", async (req, res) => {
13
14   try {
15
16     const {
17       name,
18       email,
19       password,
20       role,
21       profile
22     } = req.body;
23
24     // CHECK USER
25
26     const [existingUser] = await db.query(
27       "SELECT * FROM users WHERE email = ?",
28       [email]
29     );
30
31     // USER EXISTS
32
33     if (existingUser.length > 0) {
```

Figure 12: API Routes

Dashboard.js:

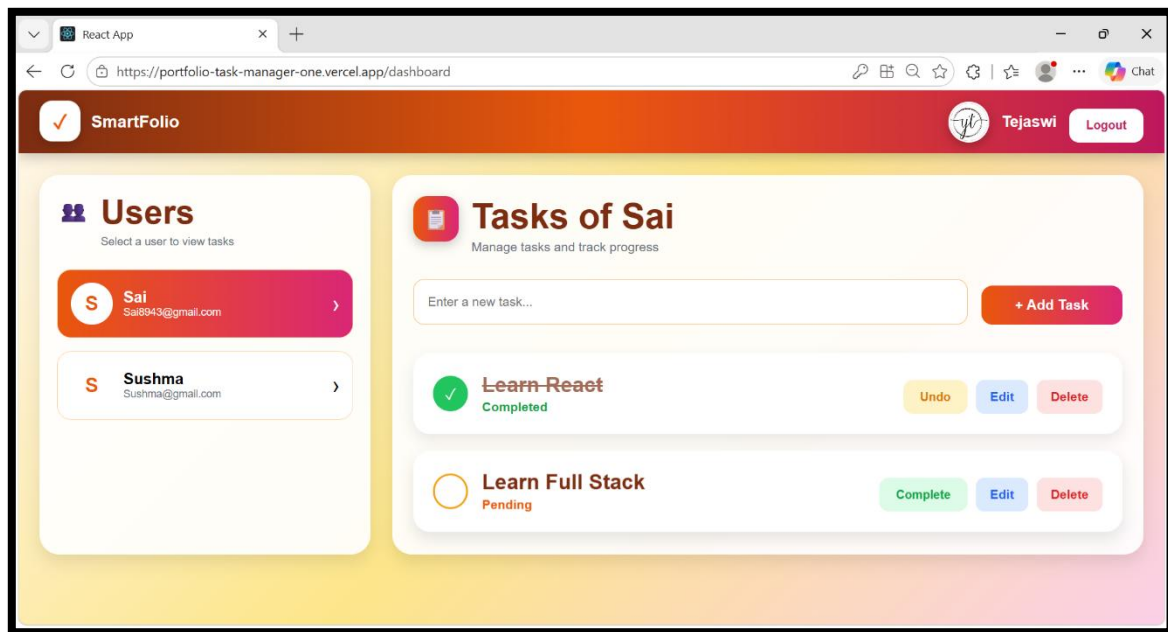


Figure 13: Tasks Dashboard

StudentDashboard.js:

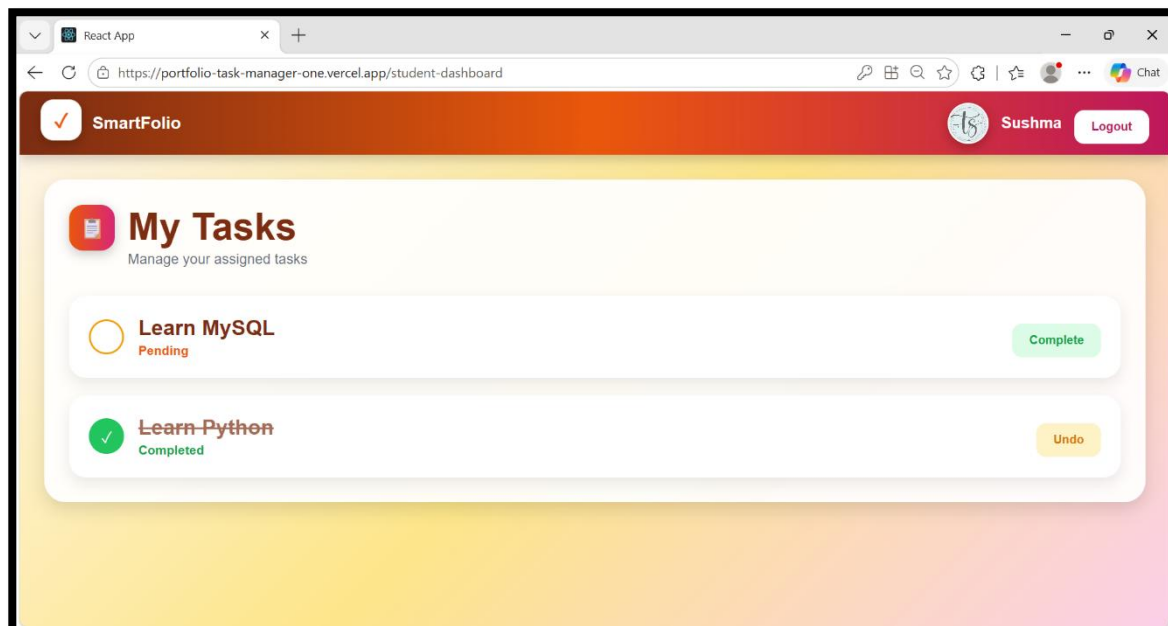


Figure 14: Student Tasks Dashboard

9. DATABASE CONNECTIVITY

Database connectivity in the Personal Portfolio & Task Management Web Application has been made possible by using MySQL with backend developed in Node.js & Express.js. The connection with the MySQL database was made from the backend server by utilizing the database connectivity modules and there was a flawless communication between the application and the data stored in the database. This connection allows the application to effectively store, fetch, modify and delete the task-related data in the system.

SQL queries are sent from the backend server to the MySQL database whenever the user carries out some action such as addition of a task, modification of task details, viewing the task and deletion of the task. Database processing of these queries returns the appropriate results to the backend server, which send these results to the frontend in the form of JSON objects. The reliable database connectivity will ensure safety of data management and efficient performance of application for the CRUD operation.

Database connection:

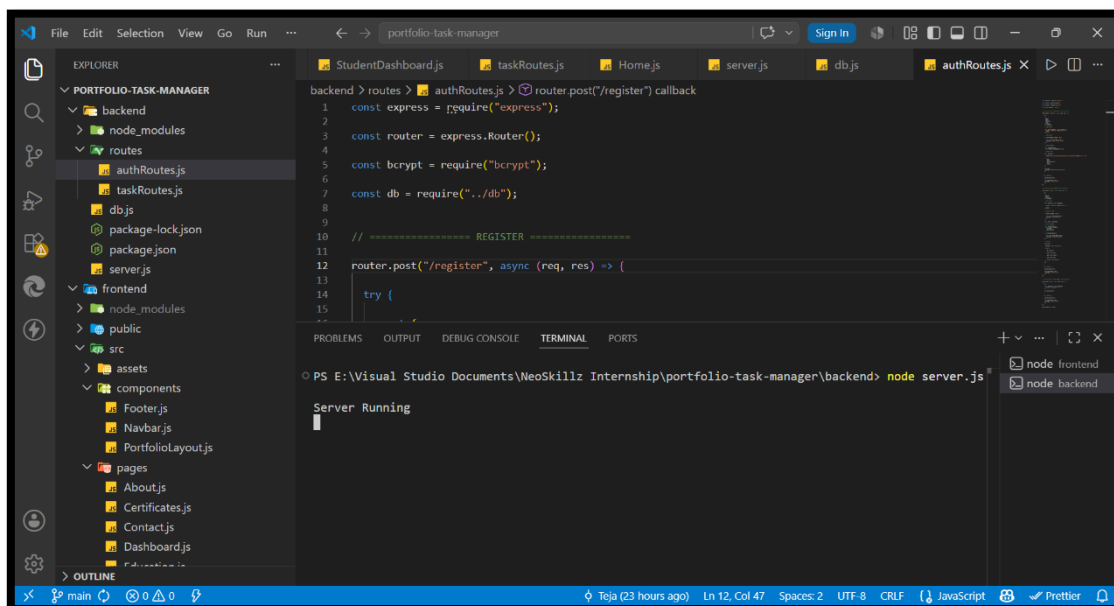


Figure 15: MySQL Database connection

MySQL Tables:

Users table:

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Schemas' tree with 'portfolio_task_manager' selected. The main editor shows the 'Query 1' window with the following SQL code:

```
6 id INT PRIMARY KEY AUTO_INCREMENT,  
7  
8 name VARCHAR(100) NOT NULL,  
9  
10 email VARCHAR(100) UNIQUE NOT NULL,  
11  
12 password VARCHAR(255) NOT NULL  
13 );  
14  
15  
16 select * from portfolio_task_manager.users;  
17
```

The 'Result Grid' shows the data for the 'users' table:

id	name	email	password	role	profile
1	Tejaswi	tejaswi@gmail.com	\$2b\$10\$UJ2R.Qmuyvutobw/.ZhcInRm7...	admin	data:image/jpeg;base64,/9j/4AAQSkZJRgABAQ...
2	Sa	Sa8943@gmail.com	\$2b\$10\$F3ox4mm88t4vZmnnv5eN0PQX1DEf...	student	data:image/jpeg;base64,/9j/4AAQSkZJRgABAQ...
3	Sudma	Sudma@gmail.com	\$2b\$10\$HqZQW8V52f3p3K6g5jw3G2mHt47b...	student	data:image/jpeg;base64,/9j/4AAQSkZJRgABAQ...

The 'Output' window shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	23:04:28	select * from portfolio_task_manager.users	3 row(s) returned	0.234 sec / 0.016 sec

Figure 16: Users Table

Tasks table:

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Schemas' tree with 'portfolio_task_manager' selected. The main editor shows the 'Query 1' window with the following SQL code:

```
18 CREATE TABLE tasks (  
19  
20 id INT PRIMARY KEY AUTO_INCREMENT,  
21  
22 task VARCHAR(255) NOT NULL,  
23  
24 completed BOOLEAN DEFAULT false  
25 );  
26  
27  
28 select * from portfolio_task_manager.tasks;  
29
```

The 'Result Grid' shows the data for the 'tasks' table:

id	task	completed	user_id
1	Learn Full Stack	1	2
2	Learn MySQL	0	2
3	Learn React	0	3
4	Learn Python	1	3

The 'Output' window shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	23:04:28	select * from portfolio_task_manager.users	3 row(s) returned	0.234 sec / 0.016 sec
2	23:06:41	select * from portfolio_task_manager.tasks	4 row(s) returned	0.000 sec / 0.000 sec

Figure 17: Tasks Table

10. API TESTING USING POSTMAN

The API tests for Personal Portfolio & Task Management Web Application were conducted using Postman for ensuring proper functioning of REST API and its interactions between front-end, back-end, and database.

HTTP requests like GET, POST, PUT, DELETE are made for performing CRUD operations for tasks. Postman is used for checking API request and response from server, data validation and handling of the errors.

GET Method:

To get the details related to task, from the database-use get method. The data was sent from the API, retrieve from the database successfully, and display in JSON through postman.

POST Method:

To insert the new task details to the database, use post method. Request body contains the task details and the data was stored in the database through the API successfully.

PUT Method:

To insert the new task details to the database, use post method. Request body contains the task details and the data was stored in the database through the API successfully.

DELETE Method:

To insert the new task details to the database, use post method. Request body contains the task details and the data was stored in the database through the API successfully.

POST Method for User Registration

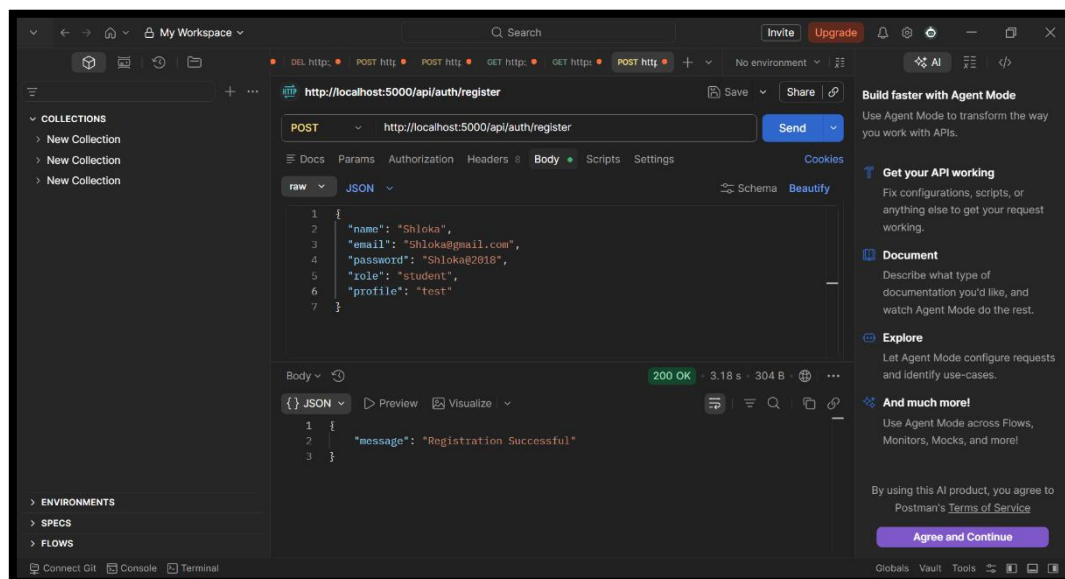


Figure 18: Testing User Registration API using POST Method

POST Method for User Login

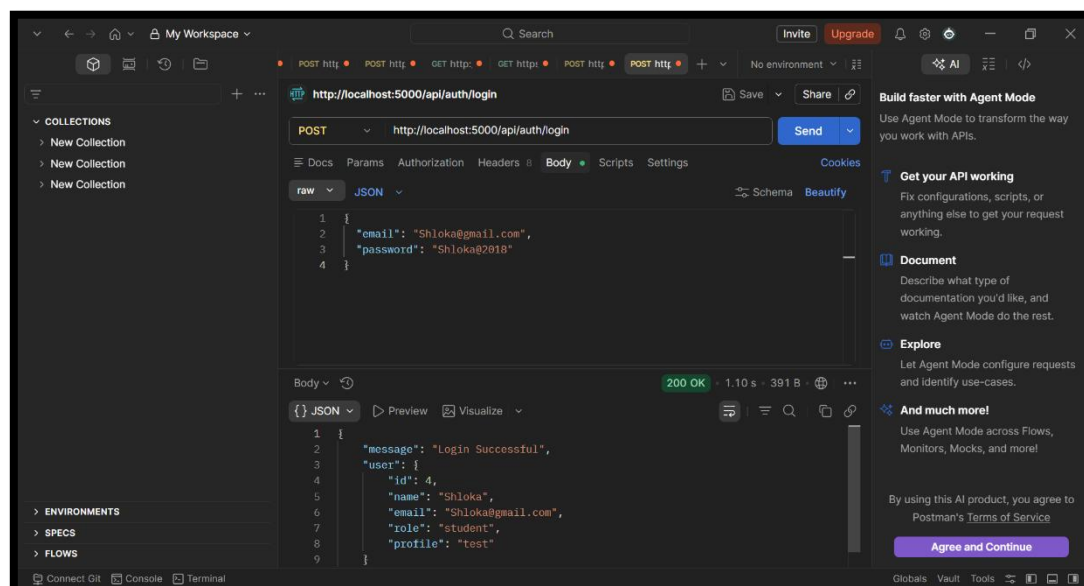


Figure 19: Testing User Login API using POST Method

The screenshot shows the Postman application interface. At the top, there's a navigation bar with the Postman logo, navigation icons, and the text "My Workspace". Below this is a toolbar with icons for various actions. The main workspace is divided into several sections. On the left, there's a sidebar with "COLLECTIONS" and "ENVIRONMENTS". The central area displays a REST client request for "http://localhost:5000/api/auth/users" with a "GET" method. Below the URL bar, there's a table for "Query Params" and a "Body" section showing a JSON payload. The right sidebar contains a "Build faster with Agent Mode" section, a "Get your API working" section, a "Document" section, and an "Explore" section. The bottom status bar shows "200 OK" and "1.40 s".

Build faster with Agent Mode
Use Agent Mode to transform the way you work with APIs.

Get your API working
Fix configurations, scripts, or anything else to get your request working.

Document
Describe what type of documentation you'd like, and watch Agent Mode do the rest.

Explore
Let Agent Mode configure requests and identify use-cases.

And much more!
Use Agent Mode across Flows, Monitors, Mocks, and more!

By using this AI product, you agree to [Postman's Terms of Service](#)

Agree and Continue

Globals Vault Tools

PUT Method for Password Reset



GET Method to Retrieve Tasks

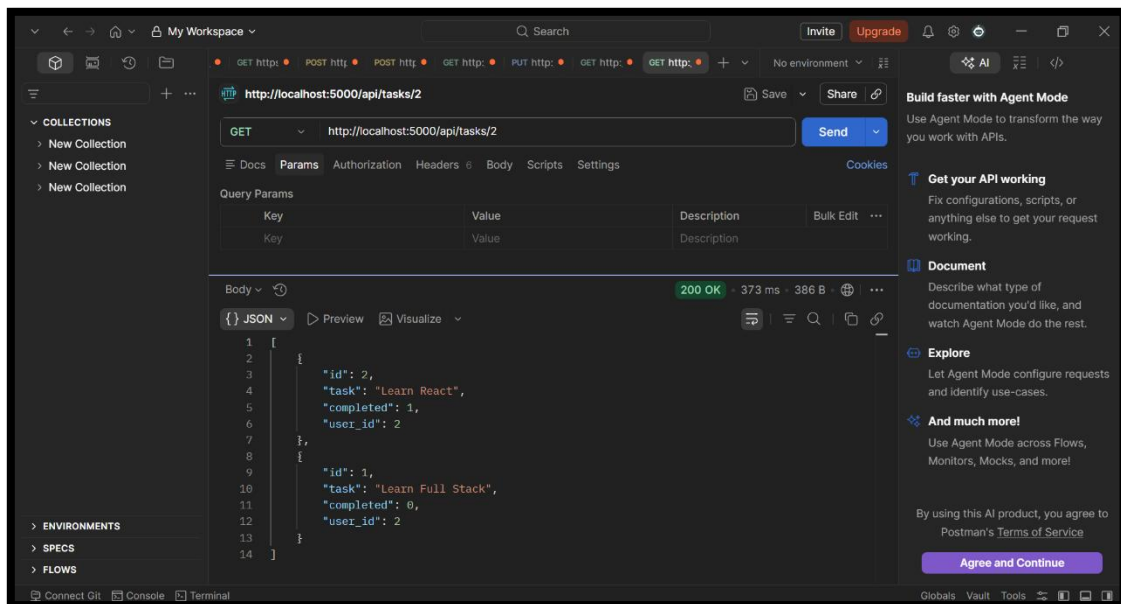


Figure 22: Retrieve Tasks

Add Task to the User using POST Method

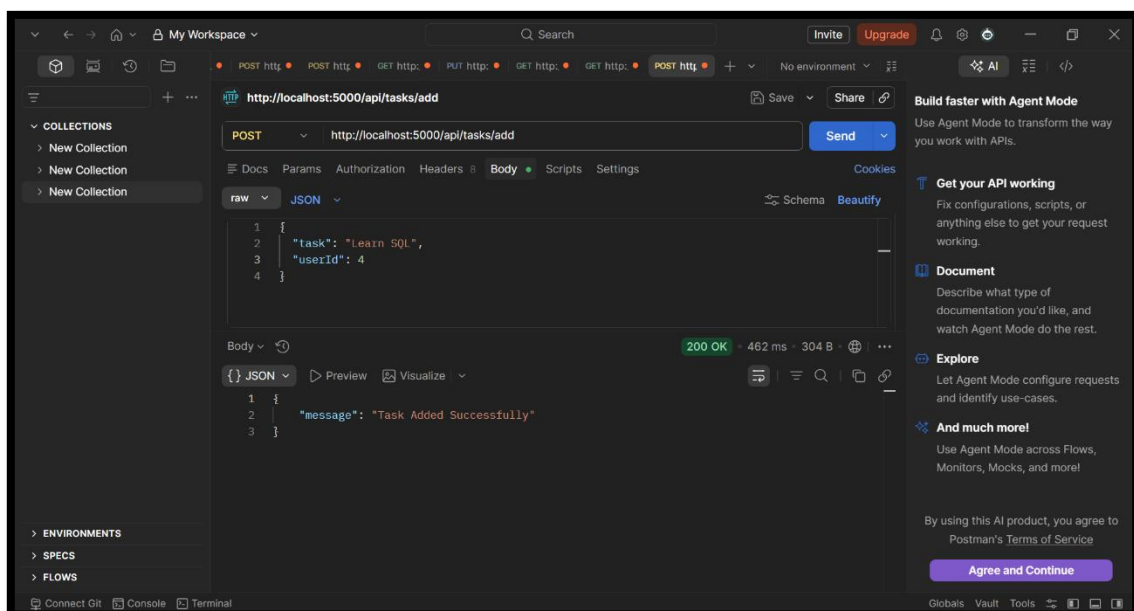


Figure 23: Add Task to the user

Update Task using PUT Method

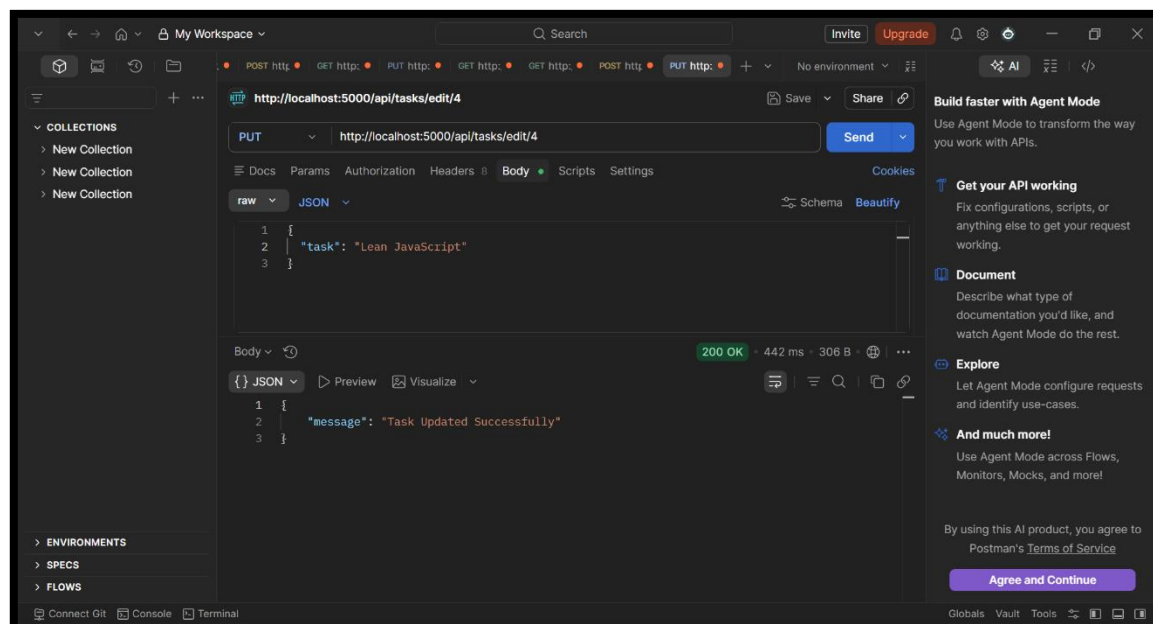


Figure 24: Update Task

Delete Task using DELETE Method

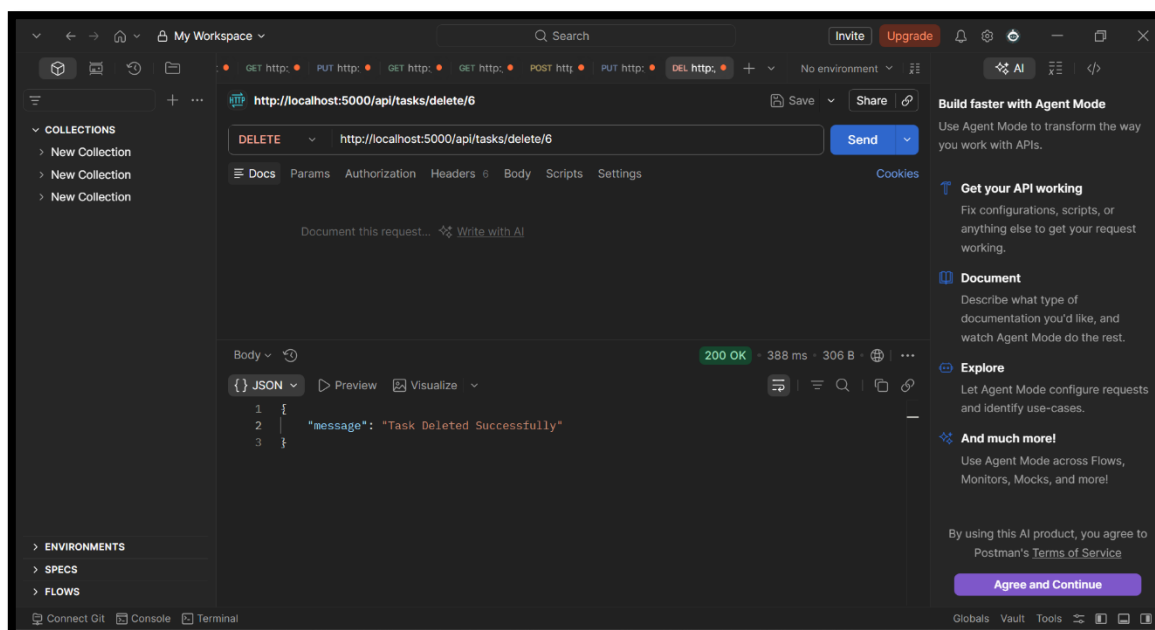
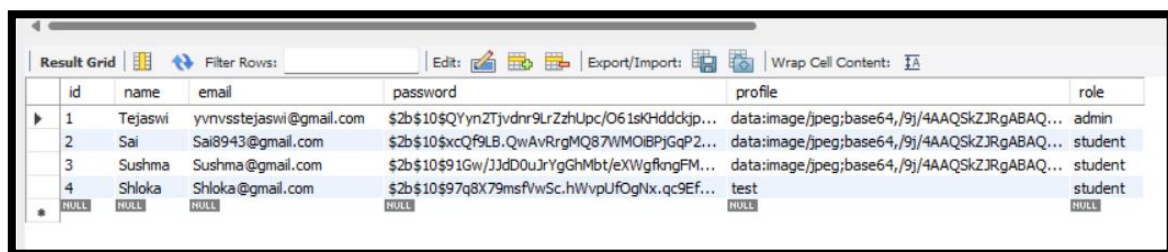


Figure 25: Delete Task

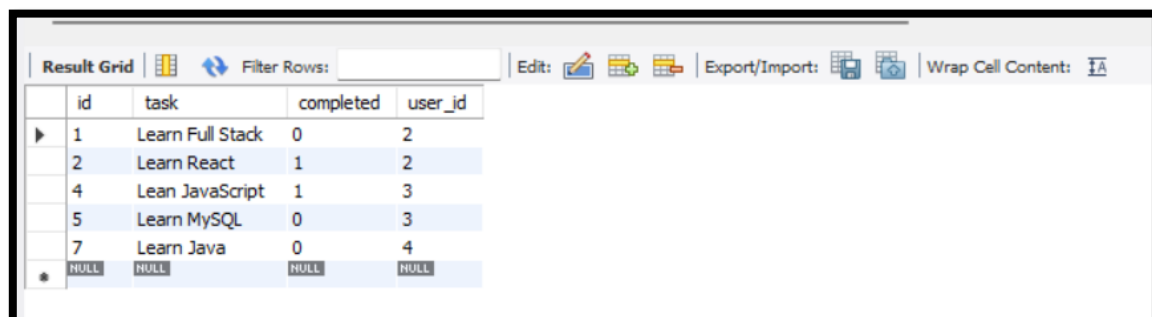
Below are the screenshots of the Personal Portfolio & Task Management Web Application MySQL database tables & dashboard interfaces. Database tables present the data that is stored, updated, and manipulated in MySQL database with respect to users & tasks. Table task represents the operations performed through application that includes adding, updating, completing, and deleting the task. Dashboard pages represent the User Interface designed by React.js and how user will perform actions. The display data on the screen get updated in real time from backend and database through integration.



A screenshot of a database table interface showing a list of users. The table has columns for id, name, email, password, profile, and role. The data is as follows:

id	name	email	password	profile	role
1	Tejaswi	yvnvsstejaswi@gmail.com	\$2b\$10\$QYyn2Tjvdnr9LrZzhUpC/O61sKHddckp...	data:image/jpeg;base64,/9j/4AAQSkZJRgABAQ...	admin
2	Sai	Sai8943@gmail.com	\$2b\$10\$xcQf9LB.QwAvRgMQ87VMOIBPjGqP2...	data:image/jpeg;base64,/9j/4AAQSkZJRgABAQ...	student
3	Sushma	Sushma@gmail.com	\$2b\$10\$91Gw/3JdD0uJrYgGhMbt/eXWgkngFM...	data:image/jpeg;base64,/9j/4AAQSkZJRgABAQ...	student
4	Shloka	Shloka@gmail.com	\$2b\$10\$97q8X79msfVwSc.hWvpUfOgNx.qc9Ef...	test	student
* NULL	NULL	NULL	NULL	NULL	NULL

Figure 27: Updated Users Table



A screenshot of a database table interface showing a list of tasks. The table has columns for id, task, completed, and user_id. The data is as follows:

id	task	completed	user_id
1	Learn Full Stack	0	2
2	Learn React	1	2
4	Learn JavaScript	1	3
5	Learn MySQL	0	3
7	Learn Java	0	4
* NULL	NULL	NULL	NULL

Figure 28: Updated Tasks Table

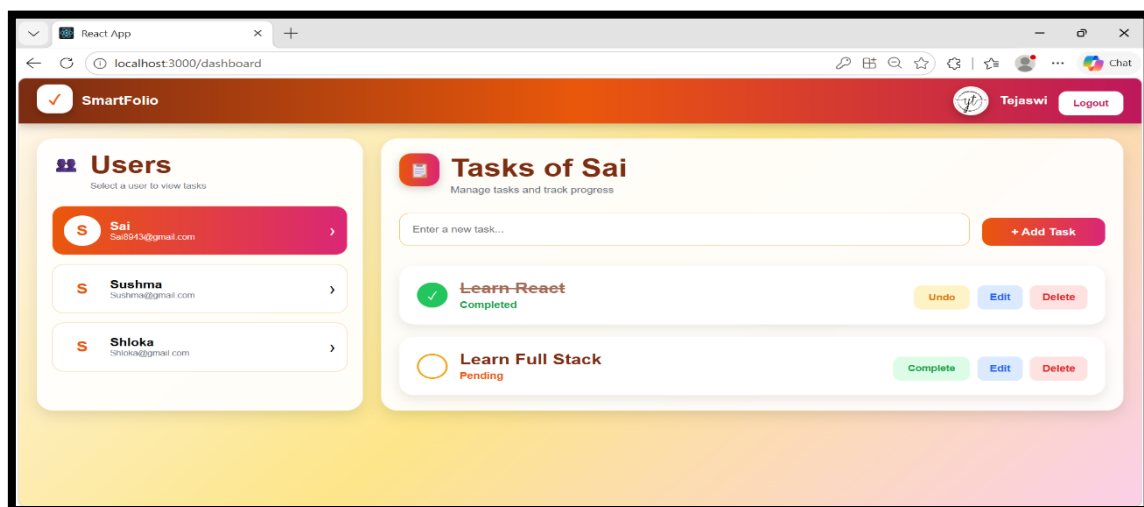


Figure 26: Task Management Dashboard Interface

11. GITHUB REPOSITORY LINK

The source code of the Personal Portfolio & Task Management Web Application was developed and managed using the GitHub for version control and the purpose of collaborating on the development process. The Git Hub was used to store the frontend and backend source files, maintain the project update, manage code changes and project structure throughout the developing phase. There is full implementation file for frontend, backend APIs, Database connection, authentication, and all the task managing functionality and deployed.

GitHub Repository Configuration

- Platform Used: GitHub
- Repository Type: Public Repository
- Version Control Tool: Git
- Repository Name: portfolio-task-manager

Repository Features

- Source code management
- Version control & tracking
- Project structure maintenance
- Code backup and updates
- Easy deployment integration
- Collaboration and sharing the project

GitHub Repository Link:

<https://github.com/yvnvsstejaswi/portfolio-task-manager>

12. DEPLOYMENT LINK

The Personal Portfolio & Task Management Web Application was deployed using new and trendy cloud hosting solutions to be accessed through the internet. Front-end for this application was deployed using Vercel a reliable and speedy hosting service for the applications built on top of React.js technology. Back-end application with REST APIs were deployed using Render for handling the server logic and the interactions with the database. Railway MySQL was utilized as a cloud database to securely store and manage user and task related information.

Deployment Configuration

- Frontend Hosting Platform: Vercel
- Backend Hosting Platform: Render
- Cloud Database: Railway MySQL

Frontend Deployment Configuration

- Build Command: npm run build
- Output Directory: build

Backend Deployment Configuration

- Service Type: Web Service
- Root Directory: backend
- Build Command: npm install
- Start Command: npm start

Deployment Link:

<https://portfolio-task-manager-one.vercel.app/>

13. TEST CASES

All the test cases were applied on the Personal Portfolio & Task Management Web Application to confirm the correct functioning of front end, backend APIs, database connection and the authentication part, task management, CRUD functionality, API interaction, and front end to Railway MySQL communication.

Table -1:

Test Case ID	Test Description	Expected Result	Status
TC-01	User Registration	User account should be created successfully	Pass
TC-02	Duplicate Registration	Application should display user already exists message	Pass
TC-03	User Login	User should login successfully	Pass
TC-04	Invalid Login	Application should display invalid credentials message	Pass
TC-05	Forgot Password	Password should update successfully	Pass
TC-06	Add New Task	Task should be added successfully	Pass
TC-07	Retrieve Tasks	Tasks should display correctly from database	Pass
TC-08	Update Task	Selected task should update successfully	Pass
TC-09	Complete Task	Task status should change to completed	Pass
TC-10	Undo Completed Task	Task should revert back successfully	Pass
TC-11	Delete Task	Selected task should delete successfully	Pass
TC-12	Database Connectivity	Railway MySQL database should connect successfully	Pass
TC-13	API Testing using Postman	APIs should return valid responses	Pass
TC-14	Responsive Dashboard UI	Dashboard should display correctly on different devices	Pass
TC-15	Deployment Testing	Application should open successfully after deployment	Pass

Figure 29: Test Cases and Results

Table -2:

POSTMAN API TEST CASES					
Test Case ID	API Endpoint	Method	Request Body / Parameters	Expected Result	Status
TC-P01	/api/auth/register	POST	{ "name": "Tejaswi", "email": "tejaswi@gmail.com", "password": "123456" }	User registered successfully	Pass
TC-P02	/api/auth/register	POST	{ "name": "Tejaswi", "email": "tejaswi@gmail.com", "password": "123456" }	User already exists message displayed	Pass
TC-P03	/api/auth/login	POST	{ "email": "tejaswi@gmail.com", "password": "123456" }	Login successful with user data	Pass
TC-P04	/api/auth/login	POST	{ "email": "tejaswi@gmail.com", "password": "wrongpass" }	Invalid credentials message displayed	Pass
TC-P05	/api/auth/forgot-password	PUT	{ "email": "tejaswi@gmail.com", "newPassword": "123456" }	Password updated successfully	Pass
TC-P06	/api/tasks/add	POST	{ "task": "Complete Project Report", "userId": 1 }	Task added successfully	Pass
TC-P07	/api/tasks/1	GET	-	Tasks retrieved successfully	Pass
TC-P08	/api/tasks/edit/1	PUT	{ "task": "Updated Project Report" }	Task updated successfully	Pass
TC-P09	/api/tasks/complete/1	PUT	-	Task marked as completed	Pass
TC-P10	/api/tasks/undo/1	PUT	-	Task reverted successfully	Pass
TC-P11	/api/tasks/delete/1	DELETE	-	Task deleted successfully	Pass
TC-P12	/api/health	GET	-	Server is running successfully	Pass
TC-P13	/invalid/endpoint	GET	-	Error response (404 Not Found)	Pass
TC-P14	Database Connection	N/A	N/A	Railway MySQL database connected	Pass

Figure 30: Postman API Testing Summary

14. CHALLENGES FACED

While building the Personal Portfolio and Task Management Web application various problems were encountered while building frontend features, backend features, the connection with the database and deployment of the application. The primary problem during the building of the application was building an appropriate communication between React.js frontend and the APIs of Node.js backend. Debugging of the API requests, error handling on the server and configuration of routes had to be carefully debugged and tested.

The second problem during the building was the connection with the Railway MySQL cloud database and appropriate configuration of environment variables to be used by the application while communicating with the database. Database connectivity errors, error handling of the CRUD operation and handling asynchronous execution of the database query was performed by changing database configuration and improving backend logic.

The next feature to be implemented was authentication features like user registration, login validation, encryption of password with bcrypt, forgot password. Handling and debugging these tasks required good testing and validation handling. Tasks were to be managed like adding, updating, deleting, completing, undoing tasks with the REST API and updating it with the frontend react component in real-time.

Deployment of frontend and backend with Vercel and Render required careful configuration of environment variables and dependencies, and various settings on the above cloud platforms. Testing with the Postman API was used to detect various routing errors and other server related issues and database responses.

These issues assisted to enhance practical knowledge related to full stack development, debugging, API handling and database administration and cloud deployment.

15. CONCLUSION

The Personal Portfolio & Task Management Web Application was developed by using full stack technology i.e. React.js as frontend and Node.js, Express.js, Railway MySQL as backend and Database. This project helps in learning knowledge related to frontend developing, create the API of backend, connecting with the database and using CRUD operations, authentication system and deployment of the web application.

The developed application is doing the following functions i.e. Signup, login, update password, create the task, update task, complete task, delete task and managing the profile on a responsive and interactive User Interface. Test the REST API through Postman to make sure all the API calls work fine with the Frontend, Backend and Database.

The development of this project has improved the technical skills about full stack web development, API integration, Debugging, Database Management, Cloud Deployment with Vercel and Render etc. In conclusion, a responsive and efficient Personal Portfolio & Task Management Web Application has been successfully developed.